AI SUPPLY CHAIN GUARDIAN / EVALUATION REPORT

Results of the entire project

End-to-end verification of detector services, Trust Engine policies, optional ML scoring, SHAP explanations, developer integrations, and PDF reporting.

Original evaluation 17 October | Latest development retest 18 October 2026 | Local deployment

729
Python passes across latest suites

9
Browser checks passed

7 / 7
Detector suites retested

OVERALL RESULT

The controlled local workflow is functional: proposals are inspected, policy decisions are persisted, supported file-tool writes are allowed or denied, and PDF reports download. This is sufficient evidence for a scoped demonstration, not unrestricted production readiness or a security guarantee.

AREA	MEASURED RESULT
Seven detectors	388 regression tests passed across latest component suites; authority-review fix and four pinned image inventories verified.
Trust Engine	219 suite passes; previously skipped PostgreSQL test now passes separately on a migrated scratch database.
Developer and analyst paths	121 utility and 9 distinct browser checks passed. Earlier isolated extension-host suite remains dated.
ML candidate	V3: 83.33% development (90); 61.11% AI pilot (18); 20.00% human-labelled upstream sample (10). Different cohorts; see pp. 31-32.
SHAP	Additive explanations verified; native/library difference 0; no observed monotonicity violations.

What these results mean

The normal website still uses weighted scoring. The new ML candidate runs separately. The fresh 83.33% development retest differs in scope/time from the historical 66.67% result and is not an independent accuracy claim. Human review of ten upstream files is now recorded; fresh weighted agreement is 30% and v3 ML replay agreement is 20%. This external weakness limits production claims.

Reading guide: historical core pp. 2-8 and metrics pp. 9-13; original figures pp. 14-20; hook follow-up pp. 21-22; latest scope-explicit retest, pinned images and new screenshots pp. 23-27; latest actual Copilot execution and first-use AI pilot pp. 28-30; submitted human review and final disposition pp. 31-32.

01 / COMPONENTS

Every detector was tested

Fresh regression results, deployment health, and the limits of what those counts establish.

DETECTOR	TESTS	LIVE CHECK / SCOPE
CipherGuard	39	Package existence, advisory evidence and package-risk contracts.
Multilingual Code Security	66	Native Python AST/syntax-tree rules; unsafe dynamic execution detected.
Secret Detection	37	Credential patterns, masking, known-example suppression and contract behavior.
Docker Security	63	Configuration/lint checks, image inventory and asynchronous result handling.
CI/CD Workflow Security	31	GitHub/GitLab workflow parsing, action references and permission risks.
License Compliance	43	Native SPDX/text matching, policy decisions and incomplete-text handling.
Prompt Injection Detection	75	Extraction, screening, semantic/judge routing and callback contracts.
Total	354	Zero failures in the final component runs.

How to interpret the pass count

These are implementation/regression tests, many with controlled network, registry, model, or LLM responses. They test intended behavior and error handling. They do not supply per-detector precision, recall, or real-world detection accuracy. Separate live fixtures were used to complement them, as described on the next page.

Source and runtime provenance

Five detector app/test snapshots matched the checked working tree by SHA256. The deployed workflow image omitted its tests and differed in readiness code/capability metadata. Secret-detection fixture copies also differed. For those two components, current-source app/tests were copied to temporary container directories and retested: workflow 31 passed; secrets 37 passed. Those replacement runs are counted once, not added to earlier attempts.

The initial workflow invocation collected no tests because they were absent from that image. The supplemental explicit run resolved test collection; it did not rebuild or change the deployed workflow process. Align the deployment image with current source before claiming an identical release build.

TOOLS ACTUALLY EXERCISED

All seven live health endpoints returned HTTP 200 / healthy. This deployment reported Guardian native code, Docker and license engines. These results are not evidence that external Semgrep, Hadolint, Trivy or ScanCode executed. Tool availability, selected backend, requested scope, and completed coverage must be stated separately.

02 / LIVE DETECTOR EVIDENCE

What the agents found

Representative inputs were synthetic or public examples. They were inspected as data; unsafe code and workflows were not executed.

AGENT / FIXTURE	OBSERVED RESULT	EXPLANATION
Code: safe addition / eval(input)	Safe: pass, full, no finding. Eval: fail, full, high unsafe_eval.	Native execution-risk detection works for this Python fixture. It is not an all-language benchmark.
Secrets: public AWS example	warn, full; low-severity aws_access_key, confidence 0.30.	The documented example is deliberately weak evidence, not a genuine leaked credential. Environment lookup passed.
CipherGuard: dependencies	Nonexistent npm package: fail, critical. Old express/lodash: warn, full; 7 advisory findings.	Checks use registry/advisory evidence. Outcomes can change with feeds, availability and package versions.
Workflow: broad permissions + unpinned checkout	fail, full; critical permission and high unpinned-action findings.	The workflow proposal is inspected before a supported file write. Nothing is pushed or executed.
License: canonical MIT / incomplete text	MIT: pass, full, confidence 0.90. Short/unknown text: warn, partial.	Recognized license text is supported; a shortened text or one license file does not prove entire-project compliance.
Prompt: blatant injection	fail, full; critical confidence 0.90. Safe Python: pass, full.	The safe Python contained zero prose spans. This clean result does not establish robustness on benign prose.
Docker: unsafe root/latest configuration	warn, full for requested native scope; 3 findings, risk 65.	Default native checks completed with zero engine errors. Full configured scope does not mean exhaustive image-CVE assurance.

Two asynchronous paths were followed to their final result

Container CVE path: an authenticated parent scan for alpine:3.20 changed from provisional to final in 2.44 seconds, ending Accept 100, full coverage. Evidence records 1 verified layer, 9 OS packages, completed native inventory and zero findings. Its manifest digest is retained. Scope is OS packages; this does not certify all language packages or all images.

Benign Markdown: a normal README changed from provisional to final in 15.06 seconds but remained Review 94.50, full coverage. The trained screen flagged a heading (probability 0.6636); the contextual judge did not confirm an attack. The disagreement produced prompt_injection_uncertain. This is a concrete unnecessary-review concern, not a clean pass.

Earlier direct-agent CVE calls returned pending/partial without a registered parent scan. The complete parent-scan result above supersedes that limited evidence for this specific Alpine fixture; broader image reliability remains unmeasured.

03 / TRUST ENGINE

Scores are not the whole decision

Routing, fusion, uncertainty floors, hard overrides, storage and callback behavior were checked together.



File tools run only after a final Accept in the configured Guardian hook.

LIVE CASE	FINAL RESULT	WHAT IT PROVES
Safe Python	Accept 100; full	Clean supported evidence can be accepted.
Unsafe eval	Review 78.75; full	A high-severity finding can require review without a hard Reject.
Unknown extension	Review 100; partial	Coverage policy prevents a high score from silently authorizing unsupported content.
Safe + unsafe batch	Aggregate Review 78.75	Per-file results remain distinct: safe Accept 100, unsafe Review 78.75.
Critical injection in ML candidate	Reject 0; hard override	Critical policy executes before model inference; the model is bypassed.

Regression and persistence results

219 unique Trust Engine tests passed. The main service-directory run passed 218 with one skipped PostgreSQL case. That case then passed against a separately migrated scratch PostgreSQL database and the scratch database was dropped. It checked state loading across store instances and concurrent callback serialization. No operational database was replaced.

The suite includes routing (49), fusion (36), callback handling (20), state behavior (20), API analysis (12), access checks (4), model seam checks (10) and other contract/integration cases. Contract-copy and runtime-copy synchronization both passed. The 10 model seam cases are included in 219, not counted twice.

All four representative weighted scans were persisted and retrieved with matching identities and decisions. Mixed-batch outcomes survived retrieval. The audit endpoint returned a valid 42,471-byte PDF. Unauthenticated history, documentation and callback requests returned 401; public health returned 200.

DECISION POLICY

The normal bands are Accept at 80 or above, Review from 50 to below 80, and Reject below 50, subject to stronger policy gates. Missing, partial, pending or failed evidence can require Review. Hard overrides can force Reject. Final means the lifecycle completed; it does not mean every requested security scope succeeded.

Initial Alembic failure from the wrong working directory was resolved by running the suite from services/trust_engine. This was an invocation correction, not a code fix. Deprecation warnings remain maintenance work.

04 / ML EVALUATION

ML score and decision results

The latest native-detector synthetic pilot was retrained and re-inferred afresh. Its historical detector observations were replayed, not collected again across all 360 artifacts.

Dataset: 360 project-authored artifacts with grouped, disjoint family splits: 180 training / 90 validation / 90 test. Artifact and submitted-content hashes were validated. Test data was not used for model selection. Each test decision class has 30 examples. Labels and numerical target scores are engineering judgments, not independently reviewed production ground truth.

90-CASE TEST METRIC	WEIGHTED	XGBOOST
Exact decision accuracy	60/90 = 66.67%	60/90 = 66.67%
Mean absolute score error	25.42 trust points	19.15 trust points
False accepts: Review/Reject labeled, predicted Accept	5	5
Reject-labeled artifacts accepted	0/30	0/30



Mean absolute error in trust points; lower is better.

Confusion matrices: true rows, predicted columns

Weighted	Accept	Review	Reject
Accept	20	5	5
Review	5	20	5
Reject	0	10	20

XGBoost	Accept	Review	Reject
Accept	20	5	5
Review	5	15	10
Reject	0	5	25

Interpretation

XGBoost reduces score error by 24.66%, but does not improve aggregate decision accuracy or remove false accepts. All five false accepts were Review-labeled Markdown instructions (5/30 of that class). No Reject-labeled artifact was accepted in this test. These are small synthetic counts, not a calibrated probability of safety or a measured real-world attack miss rate.

Full-policy accuracy by artifact type (15 test cases each): code 66.67%, Compose 33.33%, dependency 100%, Dockerfile 33.33%, Markdown 66.67%, workflow 100%. These are combined policy outcomes, not standalone detector accuracies. Expanded container-CVE scope and changing feeds also complicate older synthetic labels.

OLDER CLAIM NOT USED

The repository README quotes 77.8% accuracy and zero ML false accepts from an older, different evaluation snapshot. This report uses the later native-validation data reproduced in this run: 66.67% and five false accepts. The snapshots must not be merged or presented as the same benchmark.

05 / EXPLAINABILITY

SHAP is mathematically consistent

Fresh live inference used a separate localhost Trust Engine on port 18819 with its own SQLite state and real running detector services. The first candidate was stopped afterward; a new isolated instance was later started for the research screenshots. The normal weighted deployment was unchanged.

ISOLATED LIVE CASE	SCORE / DECISION	MODEL BEHAVIOR
Safe addition	91.12 / Accept	XGBoost; persistence verified.
Unsafe eval	10.96 / Reject	XGBoost; persistence verified.
Critical injection	0.00 / Reject	Hard override; model not invoked.

Worked explanation: safe addition

Exact TreeSHAP baseline 55.409809 plus the seven group contributions below equals the raw model score 91.116951, displayed as 91.12 after rounding. The live additivity residual was 0.00000508 trust points.

FEATURE GROUP	DETECTOR SELECTED?	TRUST-POINT CONTRIBUTION
Cipherguard	No	-1.464296
Code Security	Yes	+7.854028
Docker Security	No	-5.197208
License Compliance	No	+0.650859
Prompt Injection	Yes	+32.840919
Secret Detection	Yes	+2.285283
Workflow Cicd	No	-1.262448

Groups for unselected detectors can still contribute through missingness/artifact-pattern features. Their contribution is not a finding from an agent that ran. A positive value increases this model prediction relative to its baseline; it is not independent evidence that the code is secure.

VERIFICATION	RESULT
Native TreeSHAP vs SHAP library	Maximum difference 0 in fresh training verification.
90-test replay additivity	Maximum residual 0.00001775 trust points.
Observed-feature monotonicity	0 violations across 3,360 checks.
ML safety seam	10 tests passed: fallback, checksum/schema defense, missing data, leakage protection and override behavior.

Mathematical consistency is necessary but not model validation. SHAP explains the numerical model output before final safety gates; it does not prove causation, justify a policy override, or repair weak labels. The ordinary website will not display live SHAP while its configured scorer is weighted.

06 / END-TO-END INTEGRATION

From proposal to analyst report

The clean video workspace was preserved. Evaluation fixtures were created in private evidence directories instead.

PATH	FRESH MEASURED RESULT
CLI	Safe: exit 0, Accept 100, 0.314 s. Unsafe eval: exit 2, Review 78.75, 0.230 s.
PreToolUse adapter	Safe proposal: allow in 0.263 s. Unsafe proposal: deny in 0.320 s. Unavailable backend: deny. Adapter wrote no proposed targets.
VS Code extension host	Build passed. One isolated VS Code 1.95.3 host suite passed: manual scan, finding location, scan-on-save, unavailable-service warning, deployed API and credential-clear cancellation.
Website	8 Chromium checks passed: scan/PDF/reload, stale-response isolation, client import, history export, manual refresh/errors, origin protection, narrow layout and automatic refresh.
PDF report endpoint	Fresh safe audit: 42,109 bytes; mixed-batch audit: 42,471 bytes; valid PDF signatures. Real browser download events also passed.

How the PDF is produced

Inspection stores a scan and its request ID even when a proposed file is blocked. The analyst opens that ID in website history, reviews coverage and findings, then clicks Download audit PDF or the row-level Download PDF. The server generates the report on demand. The browser saves guardian-<request_id>.pdf to its configured download directory. It is not automatically written by Copilot or the hook.

History refreshes every five seconds on page one while the tab is visible. The local demo proxy keeps its client credential outside the browser and rejects foreign-origin requests. Backend history access still requires authentication when called directly. This is loopback-only demo access, not public multi-user authentication.

What was not freshly automated in the editor

The installed VS Code 1.141.1 binary crashed during direct extension-host launch with a Snap/graphics/library startup issue. The pinned isolated 1.95.3 runtime passed after removing conflicting library variables. The live adapter was freshly exercised; the actual interactive Copilot hook flow had previously been observed on 16 October. That human observation is separate from the fresh host and adapter tests. Run one safe and one unsafe Copilot case again immediately before recording.

BOUNDARY OF PRE-ACCEPTANCE

The configured gate covers supported agent file tools. It does not intercept manual edits or inline Tab suggestions, and it is not an OS sandbox. Terminal/unknown tools and hook-control writes are deliberately denied. Restricted patch support was added on 18 October; see the follow-up. Hook disablement, editor timeouts or termination, alternate harnesses, and changes between approval and execution remain platform limits.

Timings above are single local observations with warm-cache effects, not p95 latency, throughput, load capacity or service-level guarantees.

07 / OVERALL JUDGMENT

Overall project assessment

The local system is operational as tested. The evidence supports a controlled project demonstration and further evaluation, not a blanket production-ready claim.

RESULT AREA	JUDGMENT / REQUIRED ACTION
Agent implementation	Regression checks pass. Live targeted findings work; measure precision/recall on independently reviewed external data next.
Trust and uncertainty policy	Pass for exercised gates, persisted state, callbacks, batching and authentication. A high score is never the only acceptance condition.
Weighted local demo	Ready for a scoped recording after the two manual Copilot checks. Explain Review as a blocking outcome, not as a passed file.
ML and SHAP	Functional research candidate; not promoted. Do not claim accuracy improvement, zero false accepts, or active website SHAP in weighted mode.
Container and prompt limits	CVE completion verified for one Alpine fixture; scope/feed/image reliability remains conditional. Benign README review shows false-positive pressure.
Release and production assurance	Align workflow image/source, update stale benchmark prose, expand adversarial/load testing and assess public access, secrets, retention, backup and tenant isolation before external deployment.

Recording checklist

1. Confirm the video folder is trusted, the correct Local harness is selected and Guardian is discovered.
2. Show a safe proposal allowed and an unsafe proposal denied, with the hook log and actual file state.
3. Match the scan ID to website evidence; show final/provisional and coverage explicitly.
4. Download and open the PDF; match its ID to the same scan.
5. Say "weighted scoring" for the normal deployment. Demonstrate ML separately only after explicitly starting and verifying that candidate mode.

Reproducibility and evidence

Base commit: `f7996c16fd91`, plus the uncommitted integration/UI changes captured by a source-hash manifest. No evaluation changes were pushed, no optional model was promoted, and the clean recording folder was not used for test writes. The initial temporary database and ML API were removed. A separate ML preview was then started for the research figures. Operational history contains the evaluation scans.

Evidence root: `.data/project-results-20261017/`
Manifest: `manifest.json`
Detector logs/provenance: `agents/assessment.md`
Trust tests/live results: `trust/SUMMARY.md`
ML replay and isolated API: `ml/summary.json`
CLI, hook, browser, editor and PDF checks: `interfaces/`

Commands: `python -m pytest scripts/tests -q`; service-directory pytest per component; `npm test --prefix frontend`; `npm test --prefix integrations/vscode` with the isolated test runtime. Exact environment-aware commands and initial/retry logs are retained. The 666 Python count is 354 detector + 219 Trust Engine + 93 utility cases; model seam cases and retries are not added again. Browser checks and the extension-host suite are reported separately.

08 / RESEARCH TABLES

Classification results for the paper

All metrics describe final full-pipeline decisions after policy gates on the same 90 held-out synthetic cases. They are not separate accuracy estimates for the seven detectors.

METRIC	WEIGHTED BASELINE	ML CANDIDATE
Accuracy / balanced accuracy	66.67%	66.67%
Macro precision	0.6794	0.6750
Macro recall	0.6667	0.6667
Macro / support-weighted F1	0.6698	0.6623
Micro F1	0.6667	0.6667
Multiclass MCC	0.5023	0.5071
Cohen kappa	0.5000	0.5000
Score MAE (unrounded model-score convention)	25.4167	19.1477

Per-class precision, recall and F1

METHOD	TRUE CLASS	PRECISION	RECALL	F1	SUPPORT
Weighted	Accept	0.8000	0.6667	0.7273	30
Weighted	Review	0.5714	0.6667	0.6154	30
Weighted	Reject	0.6667	0.6667	0.6667	30
ML	Accept	0.8000	0.6667	0.7273	30
ML	Review	0.6000	0.5000	0.5455	30
ML	Reject	0.6250	0.8333	0.7143	30

Recall for Reject rises from 0.6667 to 0.8333 with ML, while Review recall falls from 0.6667 to 0.5000. Accuracy is unchanged because improvements and regressions balance on this corpus. Macro F1 is slightly lower for ML. Both methods retain Accept precision of 0.8000.

DEFINITIONS AND DENOMINATORS

Precision = TP / (TP + FP); recall = TP / (TP + FN); F1 = 2PR / (P + R). Macro values average classes; support-weighted F1 equals macro F1 here because class supports are equal. Micro F1 equals accuracy for these single-label multiclass decisions. These standard definitions follow scikit-learn [R1].

09 / STUDY DESIGN

Paired comparisons and uncertainty

The corpus supports a reproducible pilot comparison, with limited inferential strength.

DESIGN ELEMENT	ACTUAL SETUP
Dataset and split	360 artifacts; 180 train / 90 validation / 90 test. No exact construction-family ID crosses splits.
Independent grouping unit	24 construction families: 12 train, 6 validation, 6 test. Every family contributes 15 variants.
Held-out family mix	One test family per artifact type: code, Compose, dependency, Dockerfile, Markdown and workflow.
Ground truth	Project-authored labels and engineering score anchors; no independent human review or external field sample.
Replayed information	Historical native-detector responses; fresh training/inference verification does not equal 360 new live scans.

Paired correctness comparison

BOTH CORRECT	WEIGHTED ONLY	ML ONLY	BOTH WRONG
55	5	5	25

Ten cases change correctness, five in each direction. The exact two-sided McNemar calculation on those ten discordant pairs gives p = 1.000. It provides no evidence of classification improvement on this corpus. Row-level independence is not met by the clustered synthetic variants, so this is a descriptive paired summary rather than confirmatory statistical evidence.

Group-resampled score-error difference

QUANTITY	ESTIMATE / INTERVAL
MAE improvement, weighted minus ML	6.2689 trust points (24.6647% relative).
Paired 95% percentile interval	0.5520 to 15.2032 trust points improvement.
Resampling plan	20,000 paired draws; seed 20261017; sample 6 whole families with replacement, retaining all 15 rows per selected family.

Only six held-out families are available, and artifact type is confounded with family. The interval reflects resampling of this constructed corpus, not confidence about real-world attacks. The accuracy-difference interval degenerates to [0, 0] because every family has equal correct counts under both methods; it must not be interpreted as certainty or equivalence.

Score-convention reconciliation

Publication MAE uses the unrounded numerical prediction with hard overrides forced to zero, reproducing the original training result 19.147734. API-rounded final scores give 19.147222 instead. The tiny difference is rounding, not a different classifier result. Main tables and intervals consistently use the unrounded benchmark convention.

10 / ABLATIONS AND ERRORS

Removal sensitivity, not causal ranking

Each row removes one detector input from a frozen candidate while retaining uncertainty gates. The model is not retrained for the missing detector.

REMOVED INPUT	ACCURACY %	MAE	FALSE ACCEPTS	TRUE ACCEPTS RETAINED
Full candidate	66.67	19.148	5	20/30
Cipherguard	55.56	25.025	5	15/30
Code Security	61.11	21.816	5	15/30
Docker Security	66.67	19.312	5	20/30
License Compliance	61.11	19.716	5	15/30
Prompt Injection	55.56	26.392	0	10/30
Secret Detection	38.89	38.653	0	0/30
Workflow Cicd	55.56	25.231	5	15/30

How to interpret this experiment

Removing an input changes both model features and policy uncertainty. For example, removing Secret Detection produces no Accept predictions: 0 Accept / 15 Review / 75 Reject. Its zero false-accept count is accompanied by rejection/review of every clean Accept-labeled artifact. This is conservative utility collapse, not evidence that removing secrets improves the system.

Removing Prompt Injection also reduces false accepts to zero, but halves retained true Accepts from 20 to 10 and lowers accuracy. The table therefore measures sensitivity to missing evidence under policy gates. It cannot establish that one agent has greater causal value or standalone detection accuracy than another.

Error analysis

OBSERVATION	RESEARCH INTERPRETATION
5 false accepts in both scorers	5/60 non-Accept-labeled cases = 8.33%; all were Review-labeled Markdown variants of a precedence-instruction pattern. They are not five independent attack families.
0 Reject-labeled cases accepted	0/30 in this finite synthetic pilot. This is not proof of a zero real-world miss rate.
70 full / 20 partial historical test responses	Historical coverage errors affect end-to-end decisions; 15 Docker error records were present in the replayed test responses.
Fresh benign Markdown review	Classifier/judge disagreement held normal documentation for review, showing unnecessary-review pressure in a live case.

External attack diversity, an independently reviewed benign set, and per-agent confusion matrices are still required before publishing broad precision/recall claims. Historical container coverage and feed conditions should be analyzed separately from current live health.

11 / EFFICIENCY PILOT

Warm local latency and resource context

A small, explicitly scoped microbenchmark was added for the paper. These numbers do not measure maximum throughput or production capacity.

MODE	N	MEDIAN ms	P95 ms	MEAN ms	MIN-MAX ms
Weighted	20	119.88	161.89	125.14	93.36-178.37
ML candidate	20	159.66	195.58	161.59	131.74-203.19

Twenty requests per mode were executed serially and interleaved after one warm-up per API: ten safe Python additions and ten unsafe eval fixtures. Timing spans localhost HTTP, routing, actual detectors, scoring and persistence through a final response. It excludes human approval, browser rendering and PDF download. These fixtures contain no prose-judge spans; registry/LLM-heavy artifacts would have different timings.

The ML candidate has a higher median in this small run. Both modes used the same live detector services, with the candidate on its own API/SQLite instance. There were no concurrent users, cold-start trials, saturation tests, repeated independent machines or service-level claims. P95 is an empirical percentile from only 20 observations.

Container memory snapshot after the run

DEPLOYED SERVICE	MEMORY USAGE / LIMIT	CPU SNAPSHOT
trust_engine	90.07MiB / 768MiB	0.28%
prompt_injection	74.29MiB / 4GiB	0.59%
docker_security	107.2MiB / 2GiB	0.53%
license_compliance	35.1MiB / 768MiB	0.23%
secret_detection	19.86MiB / 768MiB	0.25%
cipherguard	42.59MiB / 768MiB	0.27%
code_security	15.71MiB / 2GiB	0.42%
workflow_cicd	17.35MiB / 768MiB	0.27%

This is one Docker statistics snapshot, not peak memory or CPU profiling. It omits the separate host ML API and frontend processes and must not be used to claim total-system memory or model-memory overhead. Raw observations and the snapshot are retained with the research assets.

12 / PAPER AND PRESENTATION

Claims you can defend

Use these results as a transparent pilot evaluation, with the scope stated alongside each number.

PAPER-READY RESULTS PARAGRAPH

On a project-authored synthetic corpus of 360 artifacts with grouped 180/90/90 splits, the weighted and XGBoost-based Trust Engine pipelines each achieved 66.67% exact-label accuracy on 90 held-out cases. XGBoost reduced score MAE from 25.42 to 19.15 trust points (24.66%), without reducing the five false accepts. Exact TreeSHAP additivity and independent implementation agreement were verified. Separate live tests exercised pre-write decisions, asynchronous callbacks, persisted history and PDF export. These results support functional feasibility; they do not establish production detection accuracy.

PRESENTATION POINT	SHOW / SAY
Problem and contribution	Show proposal -> relevant agents -> Trust Engine -> allow/deny -> analyst report. State supported tool coverage, not universal AI interception.
Working developer path	Show real PreToolUse logs plus actual allowed/blocked file state. A chat statement or screenshot alone does not prove enforcement.
Research result	Present confusion matrices, per-class F1, MAE comparison and error analysis. Explain unchanged accuracy and false accepts.
Explainability result	Show the isolated ML dashboard and SHAP contributions. The weighted main dashboard is a different configuration.
Analyst workflow	Match one scan ID through the dashboard, findings and exported PDF.

Results not yet available

This package does not supply independent per-agent precision/recall, external-scanner comparative accuracy, calibrated class probabilities, ROC/PR-AUC, a blinded analyst/user study, adversarial coverage across real repositories, cold-start/p95-at-load capacity, or a public deployment security assessment. The trust score is not presented as a calibrated attack probability. These gaps should be reported as future work rather than filled with invented metrics.

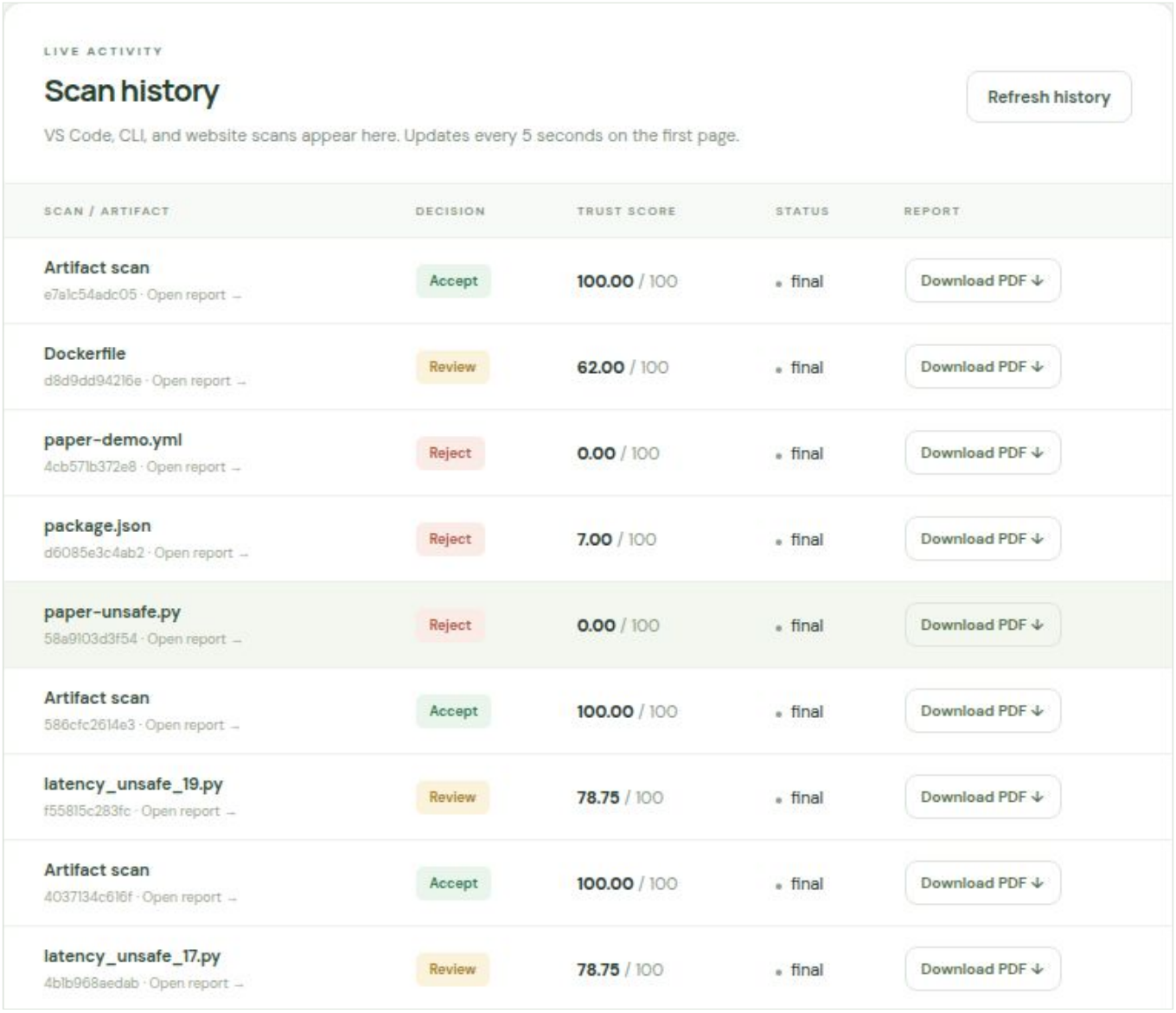
References for terminology

[R1] scikit-learn, Metrics API: classification_report, balanced_accuracy_score, matthews_corrcoef, cohen_kappa_score. Official metrics documentation.
[R2] Lundberg, S. M. and Lee, S.-I. (2017), A Unified Approach to Interpreting Model Predictions. arXiv:1705.07874. SHAP explains feature contributions to a prediction; the project-specific numerical checks are separate evidence.
[R3] Microsoft, VS Code Chat view and CLI. Official Chat documentation. CLI dispatch alone is not proof of tool execution.

13 / SCREENSHOT EVIDENCE

Actual weighted dashboard history

Fresh research screenshot fixtures were submitted through authenticated HTTP. These figures do not claim that a new Copilot session created the scans.



LIVE ACTIVITY

Scan history

VS Code, CLI, and website scans appear here. Updates every 5 seconds on the first page.

Refresh history

SCAN / ARTIFACT	DECISION	TRUST SCORE	STATUS	REPORT
Artifact scan e7a1c54adc05 · Open report →	Accept	100.00 / 100	final	Download PDF
Dockerfile d8d9dd94216e · Open report →	Review	62.00 / 100	final	Download PDF
paper-demo.yml 4cb571b372e8 · Open report →	Reject	0.00 / 100	final	Download PDF
package.json d6085e3c4ab2 · Open report →	Reject	7.00 / 100	final	Download PDF
paper-unsafe.py 58a9103d3f54 · Open report →	Reject	0.00 / 100	final	Download PDF
Artifact scan 586cfc2614e3 · Open report →	Accept	100.00 / 100	final	Download PDF
latency_unsafe_19.py f55815c283fc · Open report →	Review	78.75 / 100	final	Download PDF
Artifact scan 4037134c616f · Open report →	Accept	100.00 / 100	final	Download PDF
latency_unsafe_17.py 4b1b968aedab · Open report →	Review	78.75 / 100	final	Download PDF

Figure 1. Persisted weighted-deployment history, captured 18 October 2026. The six paper fixtures appear alongside earlier latency trials. Scores and PDF controls are the actual dashboard display.

Figure provenance

The case/scan manifest is research/figure-fixtures.json. Original unaltered full-page screenshots and their source bounds are included in the asset package. The PDF places selected visible regions of those originals for readability; it does not redraw findings or change scores. Screenshots illustrate functionality, while tables on earlier pages quantify tests and metrics.

Main configuration: localhost:3000 -> weighted Trust Engine on port 18808. Separate ML configuration: localhost:3301 -> isolated candidate API on port 18819. Keeping those configurations distinct prevents an ML screenshot from being mistaken for normal weighted operation.

14 / SOURCE-AGENT EVIDENCE

Code, secrets and prompt inspection

All three panels come from paper-unsafe.py, scan 58a9103d3f54466894d28b3380b34c18. The overall weighted result is Reject 0 by critical prompt-injection override.



Figure 2a. Native code inspection detects unsafe_eval at line 4 and provides remediation.



Figure 2b. The public AWS documentation example is retained as a low-severity hardcoding finding. It is not a real leaked key.



Figure 2c. The critical prompt-injection finding triggers an authoritative Reject. This targeted case is not an accuracy benchmark.

15 / SUPPLY-CHAIN EVIDENCE

Package and workflow inspection

The panels below are selected regions from actual weighted-deployment reports. Full original screenshots and audit data remain available.



Figure 3a. Dependency review reports seven advisory findings for old express/lodash versions. Upper portion shown; remaining findings are in the full screenshot and response. Overall score: Reject 7.00; scan d6085e3c4ab24e7fbd73575b0ea6ac5d.

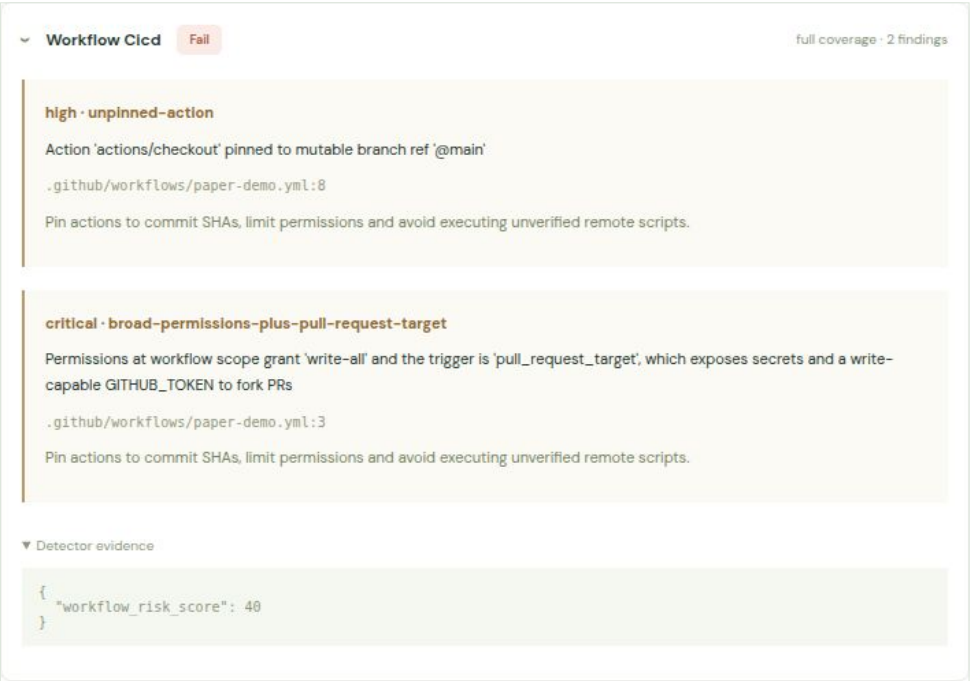


Figure 3b. Mutable checkout reference and write-all permissions under pull_request_target are reported. Overall Reject 0.00; scan 4cb571b372e84d6695fa08dc69ff75b4.

Advisory and registry findings are time-dependent. These screenshots are evidence of the observed case, not proof that every malicious or vulnerable dependency will be found. No workflow was executed.

16 / CONTAINER AND LICENSE EVIDENCE

Configuration risks and SPDX matching

Requested scope and the source of evidence remain visible to the analyst.



Figure 4a. Native container-configuration review detects root-user, latest-tag and missing-healthcheck concerns. Upper region shown. Overall Review 62.00; scan d8d9dd94216e4b8883cfbeb484fafbcd. This fixture explicitly disabled CVE/hardening options; its full coverage refers to requested configuration scope.



Figure 4b. Native SPDX evidence identifies canonical MIT text with confidence 0.90 and one compliant license. Selected region shown. Accept 100.00; scan e7a1c54adc054a99bdfd8f8382bc7a2e. Evidence explicitly limits the conclusion to supplied license text.

A separate Alpine image-CVE callback was verified in the core evaluation (page 3); the screenshot above does not claim that image-layer CVE evidence was generated for this Node configuration fixture.

17 / ACTUAL ML DASHBOARD

Prediction and explanation in the UI

This is the isolated research candidate, not the weighted main deployment. Fresh scan 2c2f5475956a48859cf2575e684f8227 uses XGBoost native-detectors-synthetic-pilot-v2.

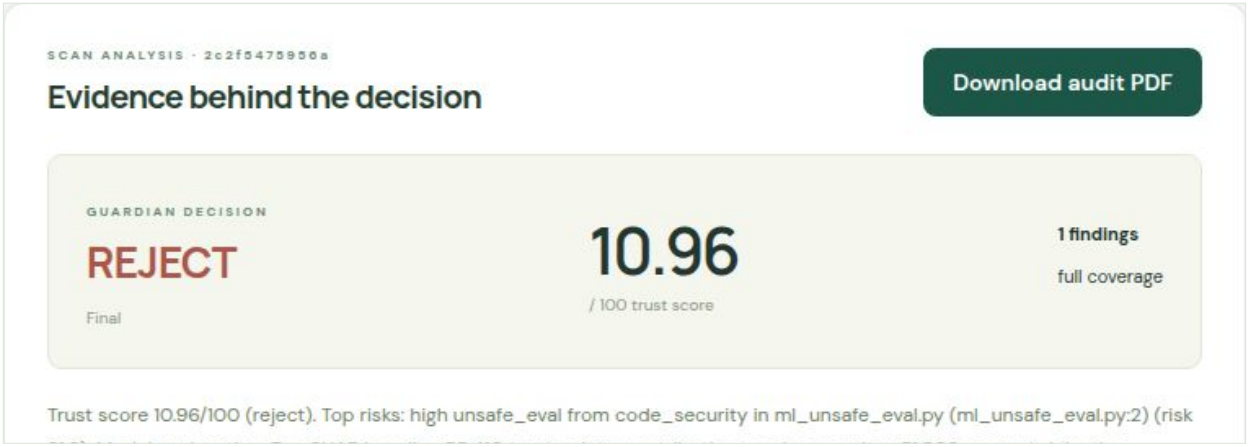


Figure 5a. Unsafe eval receives Reject 10.96 with full coverage. The detector finding and model explanation are real persisted API output.

TRUST ENGINE

Scoring: xgboost

Model: native-detectors-synthetic-pilot-v2 · project-authored synthetic pilot; no independent human review

TreeSHAP baseline: 55.41 trust points. Contributions explain the model score before safety gates.

Detector	Contribution
Cipherguard (not selected; missingness contribution)	-1.449
Code Security	-51.902
Docker Security (not selected; missingness contribution)	-5.032
License Compliance (not selected; missingness contribution)	+0.545
Prompt Injection	+12.560
Secret Detection	+2.254
Workflow Cicd (not selected; missingness contribution)	-1.423

Hard overrides and uncertainty floors remain authoritative. Model explanations are not causal evidence.

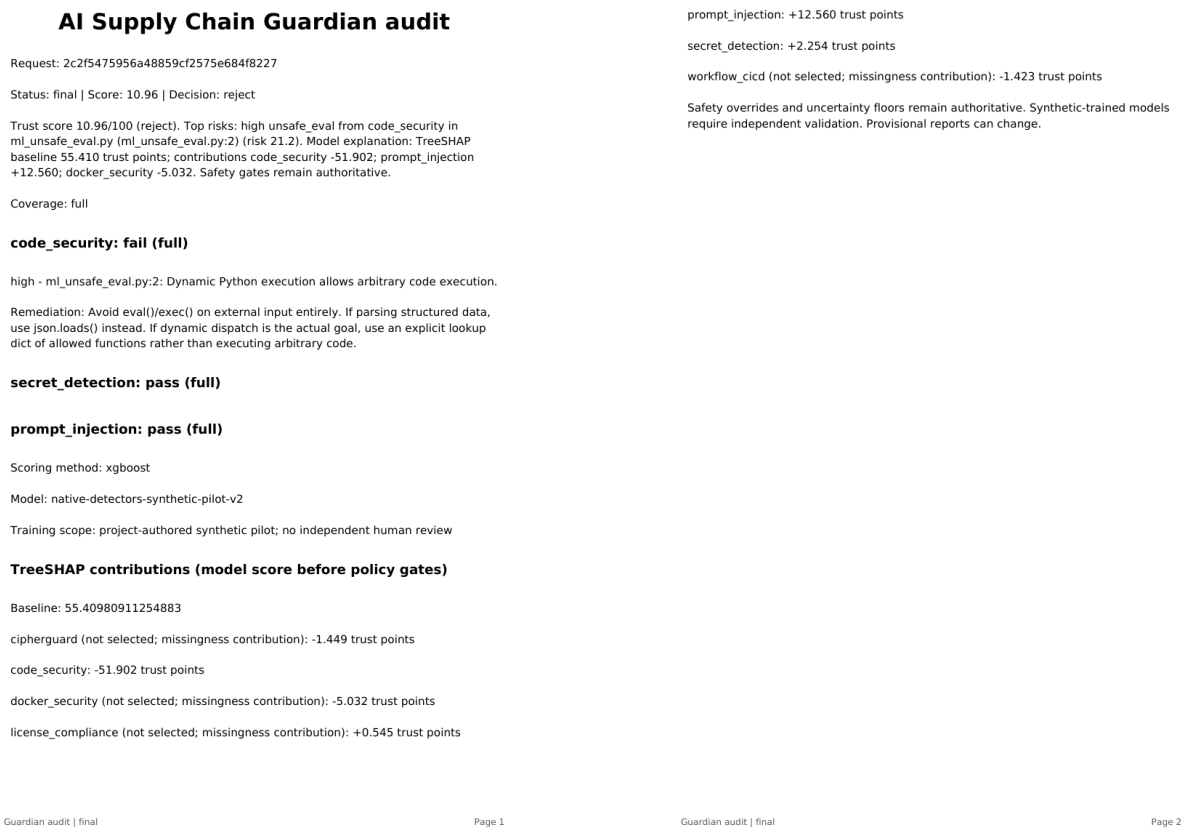
Figure 5b. Exact TreeSHAP baseline and detector-group contributions as displayed. Unselected groups are explicitly marked as missingness contributions, not findings by agents that executed.

Compare this with the weighted eval-only result Review 78.75. A more severe score on this single case does not establish better aggregate accuracy; the 90-case comparison remains 66.67% for both methods.

18 / EXPORT EVIDENCE

The downloaded ML audit report

The UI download action produced guardian-2c2f5475956a48859cf2575e684f8227.pdf in the browser Downloads directory. These are renderings of its actual two pages, not recreated report content.



AI Supply Chain Guardian audit

Request: 2c2f5475956a48859cf2575e684f8227

Status: final | Score: 10.96 | Decision: reject

Trust score 10.96/100 (reject). Top risks: high unsafe_eval from code_security in ml_unsafe_eval.py (ml_unsafe_eval.py:2) (risk 21.2). Model explanation: TreeSHAP baseline 55.410 trust points; contributions code_security -51.902; prompt_injection +12.560; docker_security -5.032. Safety gates remain authoritative.

Coverage: full

code_security: fail (full)

high - ml_unsafe_eval.py:2: Dynamic Python execution allows arbitrary code execution.

Remediation: Avoid eval()/exec() on external input entirely. If parsing structured data, use json.loads() instead. If dynamic dispatch is the actual goal, use an explicit lookup dict of allowed functions rather than executing arbitrary code.

secret_detection: pass (full)

prompt_injection: pass (full)

Scoring method: xgboost

Model: native-detectors-synthetic-pilot-v2

Training scope: project-authored synthetic pilot; no independent human review

TreeSHAP contributions (model score before policy gates)

Baseline: 55.40980911254883

cipherguard (not selected; missingness contribution): -1.449 trust points

code_security: -51.902 trust points

docker_security (not selected; missingness contribution): -5.032 trust points

license_compliance (not selected; missingness contribution): +0.545 trust points

Guardian audit | final Page 1

prompt_injection: +12.560 trust points

secret_detection: +2.254 trust points

workflow_cicd (not selected; missingness contribution): -1.423 trust points

Safety overrides and uncertainty floors remain authoritative. Synthetic-trained models require independent validation. Provisional reports can change.

Guardian audit | final Page 2

Figure 6. Exported audit pages, shown as thumbnails. The scan identity, decision and SHAP explanation match the selected ML dashboard report. The original PDF is supplied in the research asset package for readable inspection.

Traceability chain

Actual candidate API response -> persisted GET -> dashboard selection -> browser download -> PDF file. The candidate JSON and screenshot bounds are retained. PDF export is on demand; neither a Copilot chat response nor a blocked proposal automatically writes a PDF to the workspace.

A PDF is an auditable record of one scan. Its existence does not validate detector accuracy, and a provisional report may differ from a final one. Show final/provisional and coverage when using a report in a paper or presentation.

19 / EDITOR EVIDENCE STATUS

What the Copilot record establishes

A historical user-supplied editor image is included below; the new automated CLI dispatch is explicitly not counted as a completed Copilot run.

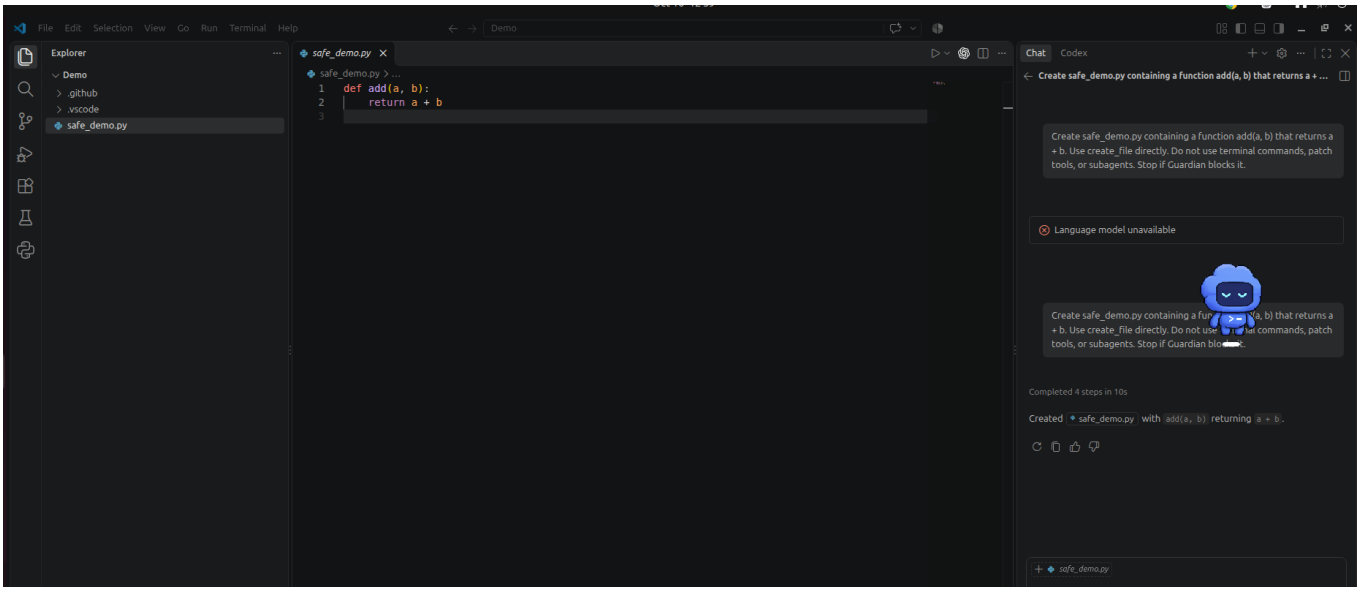


Figure 7. Original user-supplied screenshot from 16 October: safe_demo.py exists and the later chat response reports creation. The same image also contains an earlier unavailable-model attempt. The file/chat image alone is insufficient to prove gating; the retained historical PreToolUse log confirms allow at 100.0 for scan a6ffeaf33d3c407b92650e472bc28f7b.

New dispatch and capture limitation

The official VS Code CLI accepted the new six-case research prompt (exit 0), including a retry from the explicit Demo working directory. No fresh matching transcript, hook event or research target file was observed. The user then reported the chat response: create_file was unavailable, no alternative write was used, and no Guardian decision or score was produced. The proposal never reached the hook; this is not a Guardian denial. Native editor input/screenshot capture was unavailable, so no new successful Copilot execution or new editor screenshots are claimed. Fresh dashboard figures were created separately through authenticated HTTP and are labelled that way.

Complete these editor captures before publishing the demo sequence

1. In the trusted Demo folder, select Local + Agent and enable the file-edit tools. Use the updated research/copilot/prompt.txt with create or restricted apply_patch. See the 18 October compatibility follow-up below; do not bypass a denial.
2. Capture the prompt, the actual tool response, and Explorer showing the allowed file and absent blocked file.
3. Capture Output -> GitHub Copilot Chat Hooks for pre-write allow/deny. The separate Guardian output channel reports the extension's explicit/manual or save scans; it is not the hook channel.
4. Match each new scan ID to its dashboard report. Replace the historical editor illustration only after those images exist.

The exact prompt, CLI dispatch status, historical hook excerpt and missing-capture checklist are included with the research assets. No synthetic screenshot or fabricated Copilot success has been substituted.

20 / COMPATIBILITY FOLLOW-UP

Patch gate verified on 18 October

This follow-up supersedes the earlier create_file-only prompt requirement. It separates direct hook verification from an unconfirmed native Copilot run.

What changed

The adapter now accepts apply_patch as a raw patch or an input/patch field. It reconstructs Add File proposals and Update File proposals with bare @@, unique exact-context hunks in memory, then scans complete resulting files as one batch. No target is written by the adapter. Structured create/create_file and edit replacements remain supported. task_complete is now allowed as a completion notification without a detector request.

Deletes, moves, labelled hunks, ambiguous context, unsupported syntax and non-LF-terminated update targets are denied. Workspace boundaries, symlinks, control paths, file/batch size budgets, final Accept policy and stale-target checks still apply. Terminal and delegated writes remain denied. This is conservative patch support, not support for every patch grammar or a sandbox.

LIVE HOOK CASE	EXPECTED	OBSERVED	SECONDS
safe patch	allow	allow	0.325
unsafe patch	deny	deny	0.259
safe create	allow	allow	0.254
completion	allow	allow	0.124
protected	deny	deny	0.157
offline	deny	deny	0.16

Direct subprocess evidence: safe patch score 100.0, scan 335ff2cf529e4490a6ec754db3659ac1; safe create score 100.0, scan 658e9558d54b415ba14f5b0bd92635c5. Unsafe eval was blocked. All proposed target files remained absent. The completion and protected-path cases did not require a detector scan. These are actual hook/API checks, not Copilot-created files.

Fresh rerun inventory

COMPONENT	18 OCTOBER RESULT / SCOPE
Seven detectors	354 passed; same container/current-source temporary snapshot provenance as the original evaluation.
Trust Engine	218 passed, 1 skipped. PostgreSQL scratch lifecycle case was not repeated; its earlier separate pass remains dated 17 October.
Utilities / hook	111 passed, including 58 hook tests. Adds 18 cases to the original 93 utility cases; do not count retries as new tests.
Python count	683 passing cases rerun this follow-up; 684 unique passes across both days including the earlier PostgreSQL case.
Research metrics	Recomputed from the same saved historical detector/model replay inputs: accuracy 66.67% for both; score MAE 25.4167 versus 19.1477. This is not a new corpus or new 90-case live scan.

New tests cover patch reconstruction, batch policy, context ambiguity, stale targets, size/symlink/control-path defense and completion. Exact logs and live JSON are supplied.

21 / EDITOR COMPATIBILITY EVIDENCE

Tool discovery and remaining capture

The user-supplied screen below shows the actual tools configuration; it is not a generated mock-up.

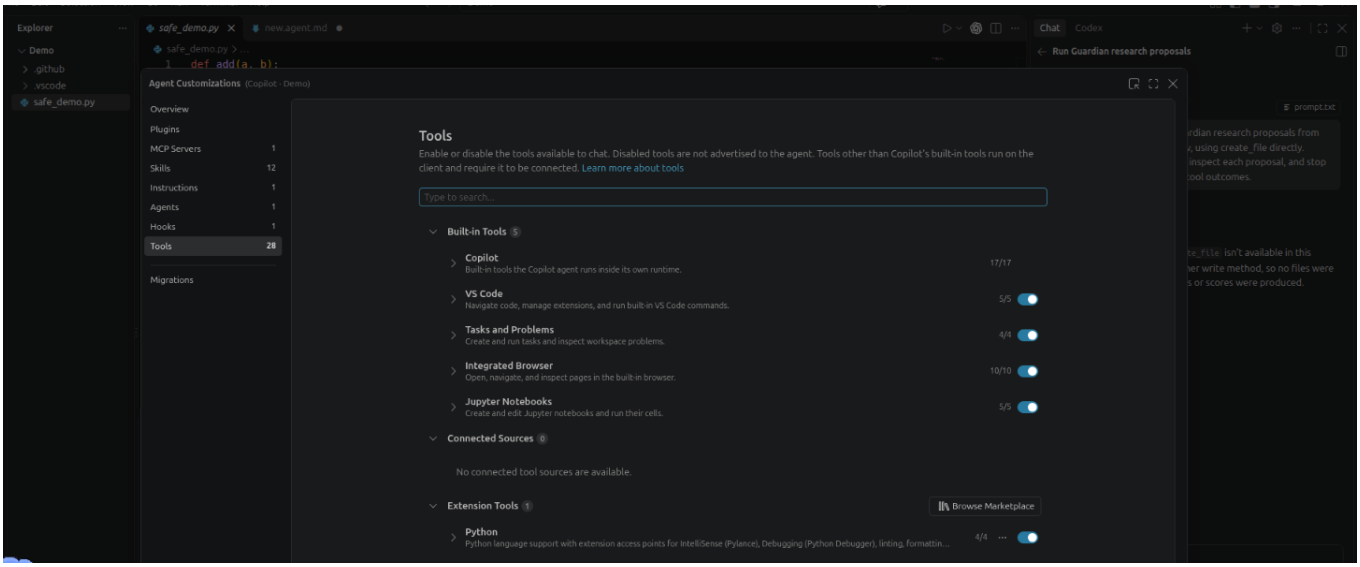


Figure 8. User-supplied 18 October tools settings: Copilot displays 17/17 enabled. Individual tool schemas are not visible in this screenshot. A subsequent read-only Copilot inspection reported apply_patch availability and repeated task_complete denials, motivating the tested compatibility fix. This image does not prove a successful patched Copilot write.

Fresh browser regression checks

Six standard authenticated/history/refresh/layout checks passed on a dedicated loopback test server; two proxy-specific checks are run separately on the configured local deployment. An initial invocation pointed at the stopped test-server port and failed its readiness wait; the corrected server run is retained separately. Results below record the actual local-proxy rerun, without hiding that setup failure.

Local proxy result: 2 passed. The two suites cover distinct browser cases and are not added to the Python count.

Copilot execution status

The workspace instructions and six-case prompt now permit create or restricted apply_patch. The updated prompt was dispatched through the official CLI, which returned zero. No fresh hook log or proposed research target was observed at report generation, so execution and new native editor screenshots remain unconfirmed. The original blocked prompt and user-reported completion loop remain historical evidence, not passing tests. No bypass writes were used.

Research and black-book interpretation

Use the existing seven detector panels, ML/TreeSHAP panels, confusion matrices, score errors, ablations, latency and PDF traceability with their stated provenance. Prompt-classifier/judge examples are NLP functional observations, not an independently labelled NLP accuracy benchmark. Per-agent precision/recall, external validation and production assurance remain unmeasured. No new accuracy, NLP, ROC or calibration claims were invented for this compatibility change.

To finish the video evidence, capture the updated prompt, actual tool response, fresh PreToolUse allow/deny log and Explorer state. Match new scan IDs to final dashboard reports and downloaded PDFs. A CLI exit code or tool-settings screenshot cannot replace this chain.

22 / FRESH DEVELOPMENT RETEST

What changed after the 66.67% result

18 October: detector fixes, explicit container scope, fresh observations and a new candidate. This is a developer-inspected retest, not a new independent benchmark.

The original 66.67% native-v2 result on pages 4 and 9 remains a historical snapshot. The new collection retains the same 360 authored artifacts and original grouped 180/90/90 split, but the old errors were examined during fixes. Container labels are now explicitly tested against supplied text, with resolved-image checks reported separately. These scope/time changes prevent a pure before/after model-improvement claim.

SAME FRESH 90 DEVELOPMENT CASES	ACCURACY	SCORE MAE	FALSE ACCEPTS
Weighted	77.78% (70/90)	19.5833	0
Frozen native-v2 ML	83.33% (75/90)	14.2267	0
Retrained v3 ML	83.33% (75/90)	13.1105	0

Frozen and retrained ML reach the same classification accuracy on the fresh inputs. Retraining improves numerical error, not label accuracy over that frozen comparator. The candidate remains separately configured; normal scoring stays weighted.

Fresh weighted final decisions

TRUE / PREDICTED	ACCEPT	REVIEW	REJECT
Accept	25	5	0
Review	0	30	0
Reject	0	15	15

Fresh retrained ML final decisions

TRUE / PREDICTED	ACCEPT	REVIEW	REJECT
Accept	25	5	0
Review	0	30	0
Reject	0	10	20

Remaining errors

The known Review-labelled precedence instruction now receives Review. However, five benign Accept-labelled Markdown variants still receive Review from screening/context uncertainty. Ten Reject-labelled code/Compose variants are Review under the new ML candidate. Review still blocks the configured write gate, but these remain exact-label errors. Zero observed false accepts on this small constructed retest is not zero real-world risk.

All 360 final healthy-collection records are retained, with no detector status error. An earlier missing-model-path setup run is excluded. Twenty-five concurrent submission-capacity rejections were completed through separate retry records. No label was changed to match a prediction.

23 / IMAGE AND EXTERNAL VERIFICATION

Pinned inputs, scope and human review

Configuration text and resolved image security have separate denominators and label requirements.

PINNED IMAGE	RESULT / COVERAGE	LAYERS	OS PACKAGES
alpine-3.20	Accept / full	1	9
debian-bookworm-slim	Reject / full	1	63
python-3.12-slim	Reject / full	4	61
node-22-slim	Reject / full	5	63

All four digest-pinned inventory runs completed without scanner errors. Alpine was accepted; Debian, Python and Node were rejected with vulnerability findings. Rejection is not an execution failure. Layers were verified without executing images. Coverage is supported OS inventory only; language packages require supplied manifests/lockfiles. Individual advisory correctness was not independently adjudicated.

Real upstream observation set

Ten real source/license files from two upstream repositories were fetched at full commit hashes, checked against retained license notices and scanned without importing/installing/executing their code. They were excluded from training. Security labels remain unreviewed; therefore there is no external accuracy claim.

UPSTREAM	LICENSE IDENTIFICATION	PINNED COMMIT
pypa/sampleproject	MIT	621e4974ca25ce53
pallets/flask	BSD-3-Clause	d086db856be18725

Full commit IDs, source URLs, hashes and license notices accompany the artifacts. License identification is not security ground truth or a transitive dependency-license audit. Three external admission rejections were retried; all ten observations finalized.

Implementation verification

SUITE	FINAL RESULT
Seven detectors	379 passed: Code 66, Docker 77, Secrets 37, Workflow 31, License 43, CipherGuard 39, Prompt 86.
Trust Engine	219 passed, 1 skipped; the separately verified earlier PostgreSQL case was not repeated.
Utilities	116 passed; includes review-integrity and hash checks.
Python aggregate	714 passed this iteration; 1 skipped. Retries and ML seam tests are not counted twice.
Browser / build	9 distinct browser checks passed across standard/local-proxy/scope runs; TypeScript/Vite build passed.

Retained logs include the corrected scope-test locator. Secret/workflow suites retain current-source temporary-snapshot provenance.

24 / FRESH LIVE DASHBOARD EVIDENCE

The missed instruction now receives Review

These are actual browser captures after the local detector/scope fixes, not reconstructed UI or new Copilot evidence.

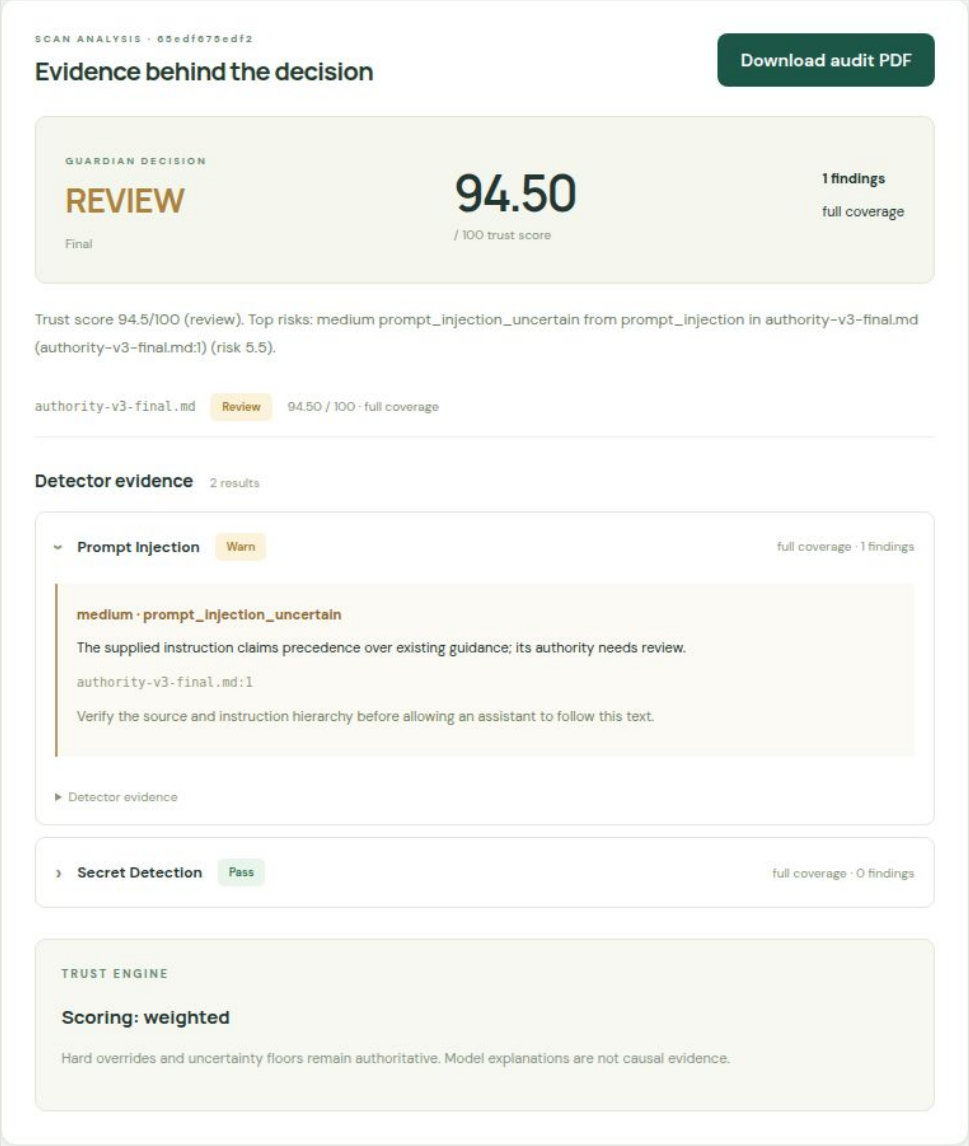


Figure 9a. The authority-precedence fixture receives final Review 94.50 in the normal weighted deployment. The detector requires authority review; it does not claim confirmed exfiltration.

The relational authority rule requires review rather than asserting a confirmed attack. Regression cases include alternative precedence wording, negated warnings and ordinary trust-store prose. This fixes the observed missed instruction while the benign Markdown screening false positives remain reported.

25 / VISIBLE SCAN SCOPE

Configuration-only is explicitly labelled

The website selector defaults to configuration plus resolved images. The narrower text scope must be selected explicitly.

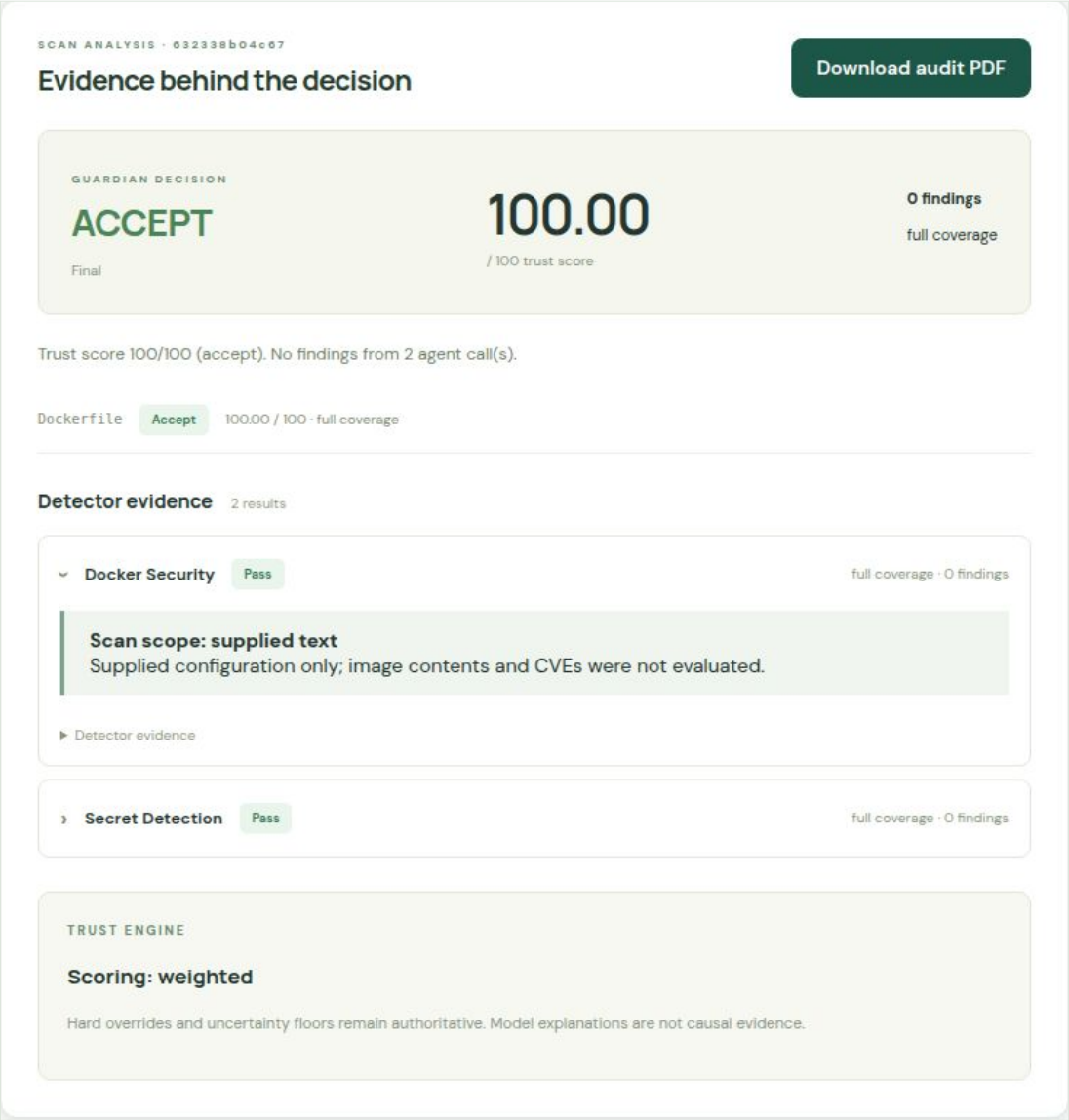


Figure 9b. A clean Dockerfile receives Accept 100.00 under explicitly selected supplied-text scope. The visible report states that image contents/CVEs were not evaluated. The actual downloaded audit PDF repeats this scope. This does not replace the separate digest-pinned image checks.

The actual scope-labelled audit was downloaded through the UI. Full coverage here means completed supplied-text checks; it does not assert that the base image is free of vulnerabilities. The four separate pinned-image observations preserve that distinction.

26 / NEW CANDIDATE AND FINAL LIMITS

Current ML prediction and exact TreeSHAP

Isolated research preview: scope-explicit-development-v3-ready. Normal weighted configuration was not promoted to this model.



Figure 10a. The actual unsafe eval proposal receives final Reject 11.45 with full requested coverage. This is a website/API fixture, not a claimed new Copilot write.

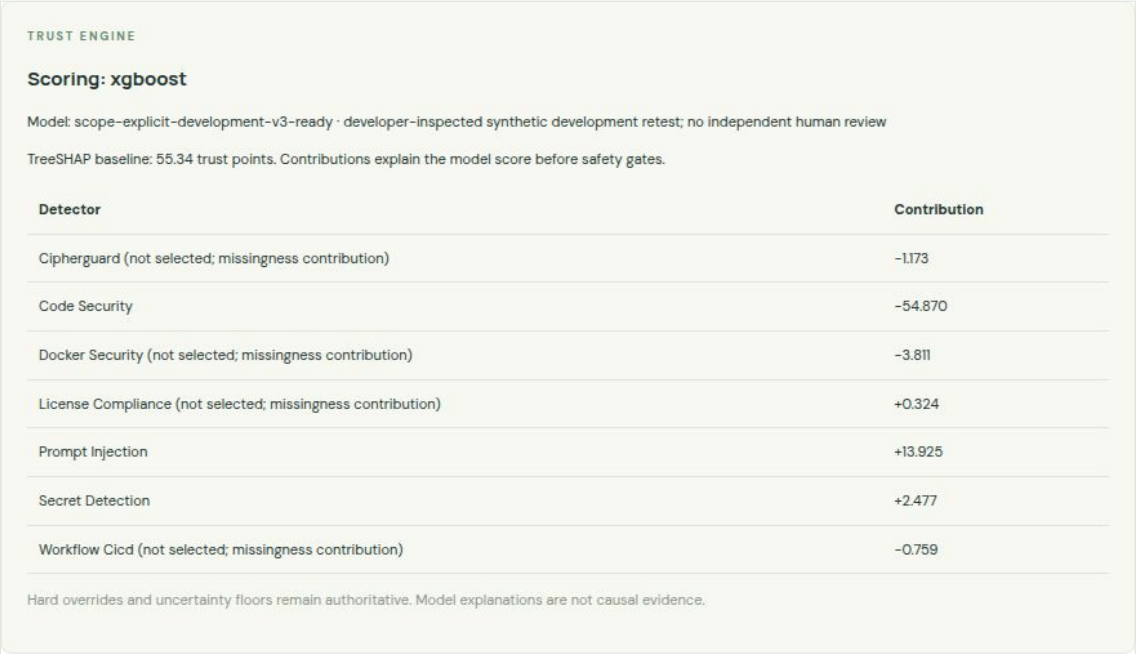


Figure 10b. Actual candidate baseline and grouped exact TreeSHAP. Unselected groups are labelled missingness contributions, not detector findings.

Verification and interpretation

Native/library SHAP difference was 0 on the checked matrix; 0 violations occurred in 3,360 monotonic probes. Four live candidate cases matched their persisted API snapshots with additivity residuals below 0.000024 trust points. These are arithmetic/integration checks, not calibrated security accuracy.

The reviewed-label step remains open. Remaining limits include the benign Markdown Review cases, previously inspected test families, only two external source projects, absent external attack labels, remote-judge/feed variability, supported OS inventory bounds, no independent NLP accuracy result and no production-load assurance. The report preserves the old benchmark rather than relabelling it to make scores look better.

LATEST / 18 OCTOBER / REAL COPILOT EVIDENCE

Pre-acceptance execution verified

Actual VS Code Copilot tool calls, saved scan records and downloaded audit PDFs.

Proposal	Saved scan / outcome
Safe Python addition	5a0d347495014e28b0fe7e5eb9b10fd0 Accept 100/100; file was created.
Unsafe eval proposal	538450cf12bb4e9c9481fdc310b2d75c Review 78.75/100; hook denied the write; file remained absent.
Version match	Both hook audit records carry adapter SHA256 5ee59a843e583003929b88ffed6bc4eff96edd52ac01864082a70816c9269abf.
Execution source	Copilot session 9b15d2ff-679a-40ad-81db-6c9dfb78f405; workspace /home/cipher/Demo/final-evidence.
Supporting checks	121 utility tests passed, including 63 hook adapter tests. The previously skipped PostgreSQL restart/distributed-lock test passed separately against a migrated scratch database; that database was removed afterwards.

Observed integration fix

The VS Code SDK exposed its apply_patch call to PreToolUse as tool_name Edit with a raw patch string. The adapter now recognizes this observed representation while preserving structured Edit handling and the final-Accept write gate. A private, permission-restricted audit journal records decisions and scan IDs without proposed content or credentials.

Evidence boundary

The records prove these two actual integration cases. They do not prove that all editing tools or future proposals are safe. Review is a denied write under this configuration; it is not labelled Reject. No native VS Code screenshots were captured. The sanitized tool transcript, audit extracts, saved response JSON and two genuine audit PDFs are in output/fresh-copilot-evidence/.

This fresh success supersedes earlier statements that fresh Copilot execution remained unconfirmed. Historical pages are retained as dated evidence, not current failure claims.

LATEST / FIRST-USE FROZEN PILOT

New cases expose remaining gaps

Labels and source/model hashes were recorded before scanning. No labels or models were changed to fit these outcomes.

Method	Exact label accuracy	Score MAE	False accepts
Weighted	66.67% (12/18)	21.74	3
Frozen v2 ML	61.11% (11/18)	19.78	3
Retrained v3 ML	61.11% (11/18)	21.11	3

The cohort contains 18 new AI-authored inert fixtures: three each for Python, Markdown, Dockerfile, Compose, workflow and package manifests. All 18 finalized with no scan failures. SHA256 checks exclude exact duplicates of the prior 360 development and ten upstream files. These are familiar attack families, not a new independent project-family sample.

Research qualification

This is a prospective AI-labelled pilot, not an independent human-labelled final benchmark. The implementing assistant authored the examples and labels. It is first-use evidence for the frozen version, but cannot establish unbiased generalization. No new training occurred. Frozen v2 and v3 model results are offline replay over the same fresh detector observations, with policy fusion reapplied; they are not additional live ML API scans.

Findings before fixes

Weak MD5 password hashing was missed when the argument had a generic name. A dependency-free package.json postinstall network-to-shell command was also missed. UNLICENSED was accepted against the AI Review label (a context-sensitive licence-policy disagreement). A benign Markdown guide received Review. Shell injection was detected but held at Review rather than the AI Reject label; a precedence instruction was rejected rather than reviewed. Blocking success and exact-label agreement are distinct metrics.

Native SHAP contribution sums matched raw model predictions to maximum residual 0.000046 for v3 (0.000015 for v2). This checks arithmetic additivity, not causal explanation, calibration or model correctness.

Serial loopback scan latency: median 0.21s; p95 3.47s. Eighteen serial cases do not establish concurrent-load performance.

LATEST / REVIEW AND RELEASE STATUS

What is completed and what remains

An AI-assisted review replaces neither a personal attestation nor a second human reviewer.

Existing upstream sample	AI decision / reason
PyPA entry point, arithmetic, unit test	Accept: literal output, arithmetic and fixed unit-test assertions.
PyPA README and MIT licence	Accept in supplied-text scope; normal documentation and retained licence notice.
Flask application factory	Review: default SECRET_KEY=dev with an optional, silently missing configuration override.
Flask database helper	Accept: application-configured database and packaged schema resource; no supplied user SQL.
Flask authentication example	Review: parameterized queries and hashed passwords; state-changing forms lack visible CSRF protection in the supplied file.
Flask README and BSD licence	Accept for supplied text. Whole-repository security and transitive licence clearance are outside this review.

All ten files were read directly without triggering new Guardian scans for that review. Review identity is Codex AI assistant; human_reviewed=false and independent_review=false. Original blank human forms and attestations remain untouched. Existing upstream predictions had already been collected historically, so this is not a formally blinded review. Eight AI Accept labels and two AI Review labels are recorded with file hashes and rationales.

Fixes and development retest

Password-like function names now supply context to Python weak-hash analysis without marking ordinary checksums. Package install lifecycle network-to-shell pipelines now produce a hard block even without dependencies. Full changed-service suites: 70 code-security and 44 CipherGuard tests passed. The same 18 cases were rescanned after deployment: weighted exact-label accuracy 77.78% (14/18), 1 false accepts against the unchanged AI labels. This is a development retest after inspecting failures; the earlier frozen results remain above.

Final status

Local execution, PDF evidence, scope-explicit scans, ML replay and SHAP arithmetic are demonstrated. Independent human label validation and a representative external holdout remain unfulfilled. The 360 old cases and this newly inspected pilot cannot be reused as untouched final evidence. Known Markdown false positives, context-sensitive licensing and exact severity-policy disagreements remain limitations. Do not claim independently validated production readiness or universally improved ML accuracy.

LATEST / SUPPLIED HUMAN REVIEW

Ten human-labelled upstream files

Review submitted by Abdulelah, dated 18 October 2026; supplied-text scope.

Validation completed

All ten sample IDs match the pinned upstream manifest, all ten SHA256 hashes match the retained file bytes, and all ten rows contain a decision, scope, rationale, reviewer identity/date and affirmative personal attestation. Labels: eight Accept, two Review, zero Reject. Your submitted CSV and rationales are retained unchanged.

Method on same human-labelled files	Stored observations	Fresh observations
Weighted baseline	30.00% (3/10)	30.00% (3/10)
Frozen v2 ML replay	30.00% (3/10)	30.00% (3/10)
V3 candidate ML replay	20.00% (2/10)	20.00% (2/10)

All ten fresh requests finalized without scan failures; saved responses matched retrieval, and all ten audit PDFs downloaded with the correct scan ID verified in their text. Candidate model figures use offline replay with policy fusion over the same detector observations, not ten additional live ML API requests. No training, threshold adjustment or label change was performed during this review evaluation.

What this review establishes

The submitted record is evidence of a project-team human review. Independence is attested by the reviewer; the script does not authenticate identity, expertise or non-involvement in earlier label construction. AI review suggestions were already available in this project conversation, so this is not a blinded study. The original files were scanned before these labels were received; a fresh rescan does not make them an untouched holdout.

Why the 83.33% figure differs

The earlier 83.33% was a 90-case synthetic development retest. The 61.11% result used a new 18-case AI-labelled pilot. The 20.00% v3 figure here uses ten real upstream files with user-supplied human labels. Different inputs, labels and evaluation roles produce different figures. None may be substituted for the others or pooled into an unsupported single accuracy number.

Small-sample and class limits

These files cover two upstream projects and contain no human Reject labels. Weighted scoring blocks six of eight human-Accept files (75% false-block rate within that class). It accepts neither of the two human-Review files, but this does not establish malicious-case recall or safety. The cohort is too small and selective for a production accuracy claim.

Evidence: output/human-reviewed-evaluation-20261018/ includes the original CSV, validation record, reviewed manifest, per-case comparisons, fresh raw responses and model metrics. This section supersedes earlier statements that the ten-file human review had not been received. Larger development/image reviews and a new untouched external holdout remain separate.

LATEST / CASE-LEVEL AUDIT

Disagreements and final status

Your labels are the comparison reference; disagreements are preserved, not relabelled away.

File	Human	Weighted	V3 ML
PyPA entry point	Accept	Review	Review
PyPA arithmetic	Accept	Accept	Accept
PyPA unit test	Accept	Review	Review
PyPA README	Accept	Review	Review
PyPA MIT licence	Accept	Review	Reject
Flask factory	Review	Reject	Reject
Flask database helper	Accept	Review	Review
Flask authentication	Review	Review	Review
Flask README	Accept	Review	Review
Flask BSD licence	Accept	Accept	Reject

Main sources of disagreement

Prompt-injection uncertainty flags benign documentation and code comments. Canonical licence matching does not accept the retained PyPA MIT notice as an exact supported match. Python SQL analysis flags execution of a packaged schema resource without proving an attacker controls that resource. Secret detection blocks the Flask tutorial default SECRET_KEY, while the reviewer requests deployment-context assessment. These are detector/context and policy problems; they cannot be solved simply by claiming SHAP is correct.

ML and SHAP interpretation

The v3 model rejects both licence files in replay and further reduces exact agreement. It remains a research candidate, not the normal deployment scorer. Additive SHAP explanations can faithfully explain an incorrect prediction; arithmetic verification is not evidence of calibration or correctness. Keep the weighted deployment as the documented default and report this external weakness alongside the earlier synthetic figures.

Final disposition

The ten-file review and its evaluation are complete. Local workflow evidence and audit PDFs are available. The new human-labelled evidence shows excessive blocking, so a claim of independently validated production readiness is not supported. Targeted general fixes and evaluation on a new project/family holdout are still required before making that claim. Do not tune to these ten files and then describe them as untouched final evidence.